

RetailPulse: A Compact End-to-End Analytics Pipeline as a Vehicle for Data-Quality Engineering

Saurabh Mastud

Independent project

<https://github.com/SaurabhMastud/retailpulse>

Abstract—Tutorials on the modern data stack typically demonstrate one link of the chain in isolation, which leaves the failure modes that only appear between components unexercised. This paper reports on RetailPulse, a deliberately small but complete batch analytics pipeline — synthetic event generation, a raw landing zone, validating ingestion into an embedded warehouse, SQL transformation as code, workflow orchestration, and a read-only dashboard — built over seven working sessions and 55 commits as a single-developer engineering case study. The system itself is unremarkable by design; the contribution of this report is an account of its data-quality strategy and of the defects that strategy caught. We classify the project’s 107 automated checks into five layers spanning unit tests, warehouse constraint tests, cross-source agreement tests, and pipeline-health guardrails, and we document six concrete defects that reached a committed state and were subsequently detected by a specific layer, together with their root causes. A recurring pattern emerges: every defect that survived review was one where a check compared an artifact against itself rather than against an independent reference, or where no check read the produced artifact back at all. We conclude with the limitations of an $n=1$ case study and the threats to validity of the evidence presented.

Index Terms—data engineering, data quality, ELT, dbt, DuckDB, workflow orchestration, software testing, reproducible builds

I. INTRODUCTION

The components of a contemporary analytics stack are individually well documented. An event source feeds a landing zone; an ingestion process loads raw records into a warehouse; transformation-as-code tools such as dbt [4] build layered models; an orchestrator such as Apache Airflow [5] schedules the whole thing; and a thin consumption layer renders the result. Material explaining any one of these links is plentiful.

What is scarcer is an account of the defects that arise *between* the links. A generator that is correct in isolation can still produce data that makes a downstream aggregate ambiguous. A warehouse test that passes on every batch can be structurally incapable of failing. A guard that is green in a developer’s working copy can be guaranteed to fail on a fresh checkout. These are integration-level faults, and a single-component demonstration cannot surface them.

RetailPulse was built to put the entire chain under test at a scale small enough that the whole thing stays comprehensible. The domain is synthetic retail commerce — page views, cart additions and purchases — chosen because the resulting analytics (revenue per day, funnel conversion, best-selling products) are intuitive enough that a wrong answer is recognisable as wrong.

This report makes three claims, each supported by artifacts in the public repository rather than by measurement on a production system:

- 1) A complete, honestly tested pipeline of this shape fits in roughly 2,000 lines of Python and 11 SQL files, which is small enough for one developer to hold in mind and therefore small enough to reason about.
- 2) Automated checks are usefully classified by *what independent reference they compare against*, not by the tool that runs them. We give a five-layer taxonomy on that basis.
- 3) Of the six defects documented here, every one was admitted by a check that lacked such an independent reference — either comparing an artifact to itself, or never reading the produced artifact back.

Section II situates the work. Section III describes the architecture and Section IV the implementation. Section V presents the data-quality taxonomy and Section VI the defects it caught. Section VII discusses lessons and threats to validity.

II. BACKGROUND AND RELATED WORK

Dimensional modelling, and in particular the separation of conformed dimensions from fact tables, is long-established practice [1]. RetailPulse follows it in miniature: a product dimension is maintained independently of the event stream, and the marts join the dimension rather than trusting attributes carried on each event. Section VI shows that this separation is what made one class of defect detectable at all.

Automated data-quality verification at scale is addressed by systems such as Deequ, which expresses declarative constraints over datasets and verifies them as part of a pipeline [3]. The present work operates several orders of magnitude below that setting and uses dbt’s built-in schema tests plus hand-written singular tests instead, but adopts the same central premise: data constraints belong in version control and run on every load.

The warehouse is DuckDB, an embeddable analytical database [2], which removes server administration from a single-developer project while keeping the SQL dialect and the dbt integration realistic. The consumption layer uses Streamlit [6]. Synthetic user activity is drawn from a Zipf-weighted popularity distribution [7], so that a minority of simulated users generate a disproportionate share of events, as observed in real traffic. Report reproducibility follows

TABLE I
PIPELINE STAGES AND THEIR IMPLEMENTATIONS

Stage	Implementation
Event generation	Python, JSON Lines output
Landing zone	Local filesystem, append-only
Ingestion	Python validation + DuckDB upsert
Warehouse	DuckDB, single file
Transformation	dbt: 6 models, 1 seed, 3 sources
Orchestration	Airflow DAG, @daily
Consumption	Streamlit, read-only

the `SOURCE_DATE_EPOCH` convention of the Reproducible Builds project [8].

III. SYSTEM ARCHITECTURE

The pipeline is a batch ELT chain with an immutable landing zone. Stages are listed in Table I.

Data moves in one direction. The generator writes one immutable JSON Lines file per batch into `data/raw/`, named by a zero-padded UTC timestamp so that lexicographic order is chronological order. Ingestion validates each record, routes rejects to a quarantine table with the reason attached, and upserts accepted records on `event_id`. A `pipeline_runs` audit table records per-batch counts of records read, loaded, duplicated and rejected. dbt then builds a staging layer of typed views and three marts at daily grain: `daily_revenue`, `funnel_conversion` and `top_products`.

Two architectural decisions are load-bearing for the rest of this report.

Idempotent ingestion. Because ingestion upserts on a natural key, the same landing file can be loaded any number of times without changing the result. This was introduced for retry safety, but it is also what made landing-zone replay — rebuilding a deleted warehouse from raw files — cost no new loading logic at all.

Logic outside the orchestrator. The pipeline stages are plain Python functions in one module; the Airflow DAG only wires them together. The immediate motivation was that Airflow has no native Windows support and the project was developed on Windows, so a DAG holding the logic would have been neither runnable nor testable. The durable benefit is that every stage is callable from a test without an orchestrator present.

IV. IMPLEMENTATION

A. Event generation

Events are emitted in browsing *sessions* rather than as independent draws: one to four events share a user, a session identifier and clustered timestamps. This is a requirement of the funnel mart, which counts view-to-cart-to-purchase transitions per visit; independent draws would make the funnel’s denominators meaningless. Session start times are spread over a configurable window (14 days by default) so that date-grained marts have more than one date to group by. Simulated users have one or two preferred categories, and activity levels follow a Zipf weighting [7].

TABLE II
CHECK LAYERS BY INDEPENDENT REFERENCE

Layer	Compared against
Unit	An expected value stated in the test
Schema constraint	A declared invariant (unique, not null)
Cross-source	A second, independently produced artifact
Pipeline health	The pipeline’s own audit and clock
Artifact read-back	The produced output, re-parsed

B. Ingestion and quarantine

Validation checks required fields, known event types, and price/quantity sanity. Invalid records are not discarded: they are written to a quarantine table with the raw payload and the rejection reason, so that a batch which silently rejects a large fraction of its rows leaves evidence behind. The distinction between a genuine insert and a skipped duplicate is obtained from the `RETURNING` clause of the upsert rather than inferred from row counts, which also makes within-batch duplicates detectable.

C. Transformation

The staging layer types and renames raw columns and nothing else. The marts sit at the finest useful grain — per date, and per (date, product) for `top_products` — on the principle that an all-time aggregate can answer exactly one question, whereas any window can be rolled up downstream from a daily grain. A session is attributed to the date of its first event, so a visit crossing midnight is counted once.

The product dimension is a dbt seed *exported from* the generator’s catalogue rather than derived from the events. This distinction matters: a dimension built as `SELECT DISTINCT product_id, product_category FROM stg_events` agrees with the events by construction and can therefore never contradict them.

D. Consumption and reporting

The dashboard separates layout from warehouse reads, so the logic that can actually be wrong — window arithmetic, rate derivation, roll-ups — is tested against a built warehouse while the rendering gets a smoke test through Streamlit’s own test runner. Project reports are generated from the architecture document by a Python renderer and committed; the PDF creation timestamp is pinned so that a diff on the committed artifact indicates a real content change.

V. DATA-QUALITY STRATEGY

The project carries 107 automated checks: 74 Python tests (one skipped where Airflow is absent) and 33 dbt data tests. Counting them is less informative than classifying them by the reference they compare against, which is what determines the faults they are capable of detecting. Table II gives the taxonomy.

The third and fifth layers are the ones most often missing in practice, and they are where this project’s real defects were found. A cross-source check requires that two artifacts be produced by genuinely different paths — hence exporting

TABLE III
DEFECTS DETECTED, BY THE LAYER THAT CAUGHT THEM

Defect	Caught by
Per-batch product catalogue	Schema constraint
Events-only category check	Cross-source (replacement)
Seed drift on fresh checkout	Cross-source, in a clone
Missing seed in test fixture	Cross-source (cascade)
Stale warehouse	Pipeline health
Mangled report identifiers	Artifact read-back

the product dimension from the generator’s catalogue, which lets a referential test fail when an event names a product the catalogue has never heard of. An artifact read-back check requires parsing the thing that was actually produced, rather than asserting that producing it did not raise.

Pipeline-health checks are deliberately split in two. A volume check asserts that the *latest* audit row did not read a batch and then load, duplicate and reject nothing, which is the signature of a stalled loader as distinct from a legitimately empty source. A freshness check asserts that the newest event timestamp has not fallen further behind the present than the generator’s own window, which catches the opposite failure: a warehouse that is voluminous but no longer advancing.

VI. EVALUATION: DEFECTS DETECTED

This is an engineering case study, not a controlled experiment. We make no performance claims; the warehouse holds a few thousand rows and rebuilds in under a second, so throughput figures would measure nothing of interest. Instead we report, as the primary evidence, the defects that reached a committed state and were later detected, with the layer that caught each one. Table III summarises them.

Per-batch product catalogue. The generator’s catalogue was seeded with the per-run random seed, so each batch invented its own product-to-category mapping. Once the product mart was re-grained to (date, product), a single product split into duplicate rows. The composite-key constraint failed on the first multi-batch run. Root cause: reference data generated with the same seed as the random data that references it.

Events-only category check. A test asserted that each product mapped to exactly one category, comparing events against other events. It could not fail in the worst case: if every event for a product agreed on the *same wrong* category, the test passed. It was replaced by a comparison against the exported dimension, which is strictly stronger. This defect was in the test suite, not the pipeline — a check that cannot fail is a defect in its own right.

Seed drift on fresh checkout. A guard asserted that the committed seed file byte-matched a fresh export. It passed in the working copy and would have failed on every fresh clone, because the version control system was configured to rewrite line endings on checkout while the exporter writes Unix endings. It was found by cloning the repository, not by running the suite.

Missing seed in test fixture. A dashboard test fixture built models without first loading the seed. The omission was

harmless until a mart began referencing the seed, at which point it failed. The ordering rule had been duplicated across four call sites and was already wrong in one of them within hours of being introduced; it was consolidated into a single function.

Stale warehouse. The freshness check failed against a local warehouse that had not been loaded for approximately six weeks of elapsed time, and failed again two weeks later. On both occasions the resolution was to run the pipeline, not to widen the threshold. A check that can be satisfied by relaxing its own bound is not a check.

Mangled report identifiers. The report renderer stripped underscores as inline emphasis markup. In a document composed largely of `snake_case` identifiers, this corrupted every one of them across six pages, for several commits. The pre-existing test asserted only that a non-empty PDF had been written, which such a report satisfies. Extracting the rendered text and asserting on it surfaced the fault immediately. Root cause: markup rules written for prose applied to a document that is mostly identifiers.

Four of the six defects were admitted by a check that compared an artifact to itself or to nothing; the remaining two were admitted because no check read the produced artifact back. None were found by inspection.

VII. DISCUSSION

A. Lessons

Three generalisations survive the specifics. First, the question to ask of any data test is what independent artifact it compares against; if the answer is “the same table”, it verifies internal consistency and not correctness. Second, a generated artifact that is also committed must be deterministic, or its version history becomes noise — this applies equally to an exported CSV and to a rendered PDF. Third, a passing local suite is not evidence that a project runs: two of the six defects above were invisible until the repository was cloned and executed from a bare checkout.

B. Threats to validity

Several limitations bound these conclusions.

Construct validity. The defect count is the number *detected*, not the number present. Defects that no layer was capable of catching are by definition absent from Table III, and their number is unknown.

Internal validity. The author designed the system, wrote the checks and classified the defects. The taxonomy was constructed after the defects were found, so its fit to them is not independent evidence of its generality.

External validity. This is a single project at small scale with synthetic data, built by one developer. Concerns that dominate production systems — late-arriving data, schema evolution, multi-writer concurrency, cost — are absent by construction, and nothing here should be read as evidence about them.

Unexercised components. The Airflow DAG has never been executed. The development platform has no native Airflow

support, so the DAG is validated structurally: its task identifiers, dependency order, and the continued existence of every function it calls are asserted at the abstract-syntax-tree level. Everything it orchestrates does run, through the same functions, via the command-line entry point. The dashboard likewise runs only locally. There is no continuous integration, so all checks are enforced by local discipline rather than by the repository.

VIII. CONCLUSION AND FUTURE WORK

RetailPulse demonstrates that a complete batch analytics chain, honestly tested, is tractable at a scale one developer can keep in mind: approximately 2,000 lines of Python, 11 SQL files, 6 dbt models and a seed, built across 7 sessions and 55 commits. Its more transferable output is the finding that the useful axis for classifying data-quality checks is the independence of their reference, and the observation that every defect which survived review in this project was admitted by a check lacking one.

The most valuable next step is continuous integration: it is the single documented limitation with a concrete fix, and it would have caught the checkout-dependent defect on the first clean build. A secondary item is to resolve the dashboard's warehouse path at call time, so that the render checks execute in environments where no warehouse has yet been built rather than skipping.

REFERENCES

- [1] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Indianapolis, IN, USA: Wiley, 2013.
- [2] M. Raasveldt and H. Mühleisen, "DuckDB: an embeddable analytical database," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2019, pp. 1981–1984.
- [3] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.
- [4] dbt Labs, "dbt documentation." [Online]. Available: <https://docs.getdbt.com>
- [5] Apache Software Foundation, "Apache Airflow documentation." [Online]. Available: <https://airflow.apache.org/docs/>
- [6] Streamlit Inc., "Streamlit documentation." [Online]. Available: <https://docs.streamlit.io>
- [7] G. K. Zipf, *Human Behavior and the Principle of Least Effort*. Cambridge, MA, USA: Addison-Wesley, 1949.
- [8] Reproducible Builds Project, "SOURCE_DATE_EPOCH specification." [Online]. Available: <https://reproducible-builds.org/docs/source-date-epoch/>